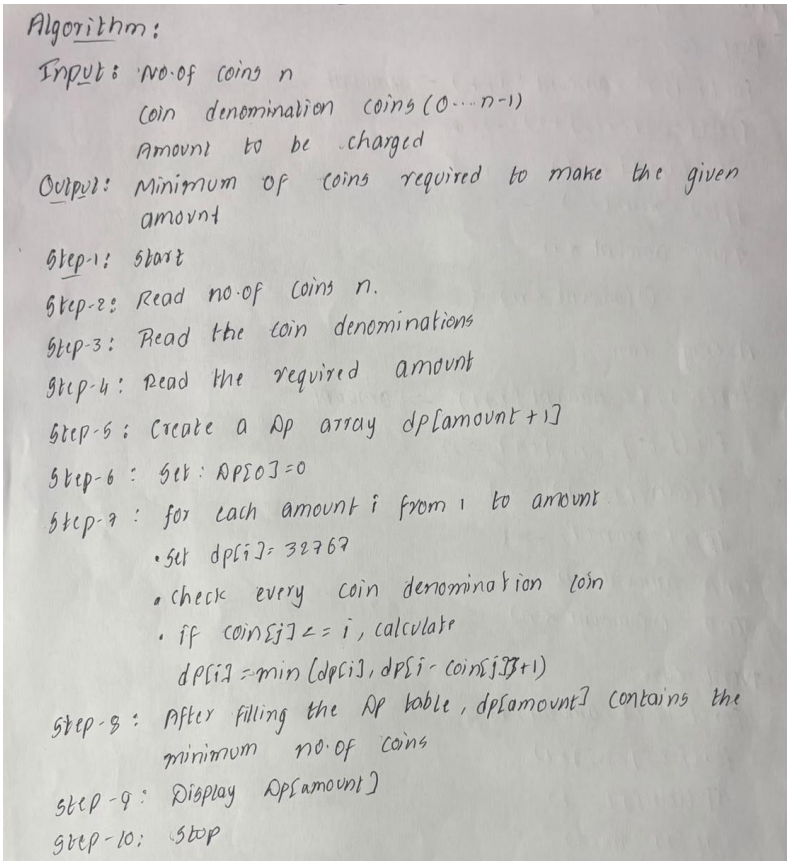
PRACTICAL – 7

AIM: Implementation of making a change problem using dynamic programming.

ALGORITHM:



Algorithm:

Input: No. of coins n
Coin denomination coins (0...n-1)
Amount to be charged

Output: Minimum of coins required to make the given amount

Step-1: Start
Step-2: Read no. of coins n.
Step-3: Read the coin denominations
Step-4: Read the required amount
Step-5: Create a Dp array dp[amount+1]
Step-6: Set: DP[0]=0
Step-7: for each amount i from 1 to amount
- Set dp[i]= 32767
- check every coin denomination coin
- if coins[j]<=i, calculate
  dp[i]=min(dp[i], dp[i-coins[j]]+1)

Step-8: After filling the Dp table, dp[amount] contains the minimum no. of coins
Step-9: Display Dp[amount]
Step-10: Stop

PROGRAM:

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
   int n, amount;

   cout << "Enter number of coins: ";
   cin >> n;

   vector<int> coin(n);
```

```
    cout << "Enter coin denominations: ";
    for (int i = 0; i < n; i++) {
        cin >> coin[i];
    }

    cout << "Enter amount: ";
    cin >> amount;

    vector<int> dp(amount + 1, 32767);

    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coin[j] <= i && dp[i - coin[j]] != 32767) {
                dp[i] = min(dp[i], dp[i - coin[j]] + 1);
            }
        }
    }

    if (dp[amount] == 32767) {
        cout << "Amount cannot be formed using the given coins."
             << endl;
    } else {
        cout << "Minimum number of coins required = "
             << dp[amount] << endl;
    }

    return 0;
}
```

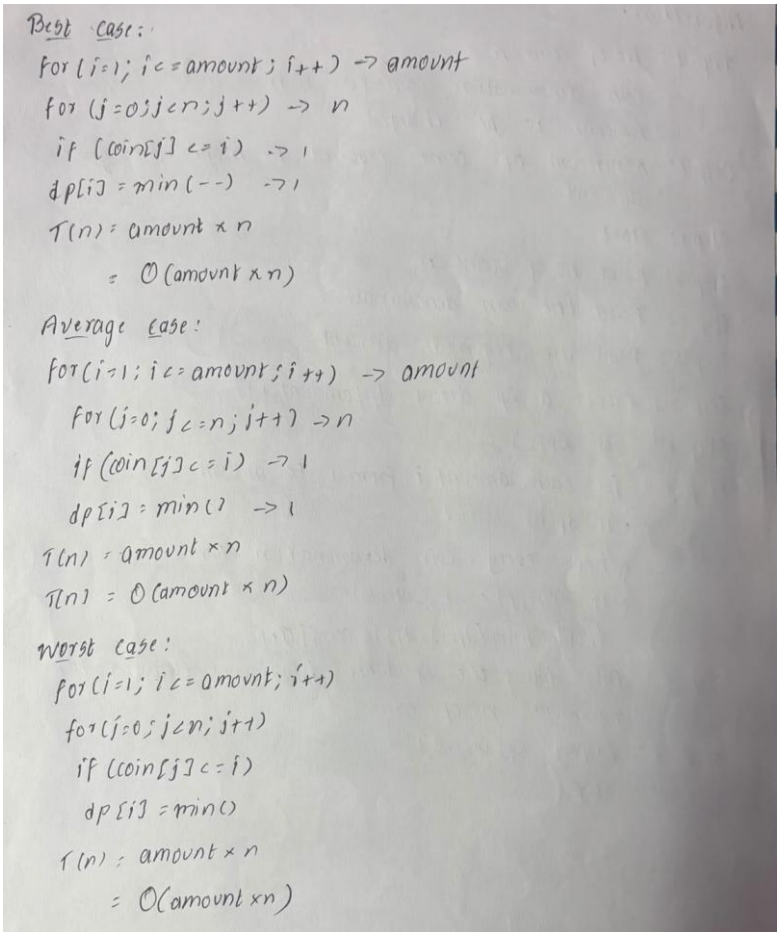
OUTPUT:

```
Output

Enter number of coins: 6
Enter coin denominations: 1 3 5 7 9 10
Enter amount: 17
Minimum number of coins required = 2


=== Code Execution Successful ===
```

TIME COMPLEXITY:



CONCLUSION:

The implementation and time analysis of the Making Change Problem using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different coin denominations to find the minimum number of coins required to make the given amount. It has a time complexity of O(n × amount) in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Making Change Problem.